

Problem Statement

Date	01 October 2026
Team ID	6ab22737f32d8c9d8b225a8a
Project Name	Implement Client Script & UI Policy (Incident)
Maximum Marks	5 Marks

Problem Statement

A problem statement clearly defines the issue your project solves, where it occurs, and what outcome is expected. Complete the sections below to document the problem, scenarios, requirements, and analysis for this project.

Step 1: Problem Overview

Field	Details
Platform / Tool Used	ServiceNow (Personal Developer Instance – PDI)
Type of Customization	Client-Side Customization – Client Script (onLoad, onChange, onSubmit) & UI Policy
Target Module / Table	Incident [incident] – Service Desk > Incidents form
Environment	ServiceNow PDI (Cloud Hosted Instance)
Problem Identified On	

Step 2: Problem Scenarios

S.No	Problem Scenario / Description	Current Behaviour (As-Is)	Expected Outcome
1	Resolution details are not enforced when an incident is moved to Resolved / Closed, so tickets are closed without proper information.	Close code and Close notes remain optional on the form until the server rejects the update.	UI Policy makes Close code and Close notes mandatory when State = Resolved or Closed.
2	Irrelevant fields are always visible and the form is cluttered; fields stay editable after closure.	All fields are shown/editable irrespective of Category or State.	UI Policy shows/hides fields based on Category and makes fields read-only when State = Closed.
3	Users submit incidents with very short or meaningless Short description values.	No client-side validation; poor-quality tickets are saved.	onSubmit Client Script blocks save and shows an error if Short description is under 10 characters.
4	Changing Category leaves a stale Subcategory value and gives no guidance to the user.	Old Subcategory persists after Category changes, causing wrong data.	onChange Client Script clears Subcategory and shows an info message; onLoad script greets/guides the user on a new incident.

Step 3: Requirements & Acceptance Results

S.No	Requirement / Check	Target Value	Actual Value	Status (Pass / Fail)	Remarks
1	UI Policy – Mandatory fields	Close code & Close notes mandatory when State = Resolved/Closed			
2	UI Policy – Field visibility / read-only	Fields hide/show by Category; read-only when Closed			
3	Client Script – onSubmit validation	Save blocked if Short description < 10 chars			
4	Client Script – onChange (Category)	Subcategory cleared and info message displayed			
5	Client Script – onLoad	Guidance message shown on new incident form			
6	Form Load / Errors	No JavaScript console errors; form loads normally			

Step 4: Observations & Analysis**Problem Description:**

[Describe the problem: Incident form lacks client-side controls, leading to incomplete, inconsistent and low-quality incident records.]

Gaps / Root Cause Identified:

[List gaps, e.g., no mandatory enforcement on resolution fields, no field visibility rules, no input validation, no dynamic behaviour on Category change.]

Proposed Solution:

[Implement UI Policies (mandatory, visible, read-only) and Client Scripts (onLoad, onChange, onSubmit) on the Incident table, then test each scenario on the form.]

Step 5: Screenshots / Evidence

Attach screenshots of your implementation below (e.g., UI Policy record, Client Script record, Incident form behaviour):

[Insert Screenshot 1 here]

[Insert Screenshot 2 here]